

ASSIGNMENT- 13.5

Name:MD ARSHAD

HT.No: 2303A51334

Batch: 20

Task Description #1 (Refactoring – Removing Global Variables)

Sample input & prompt

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

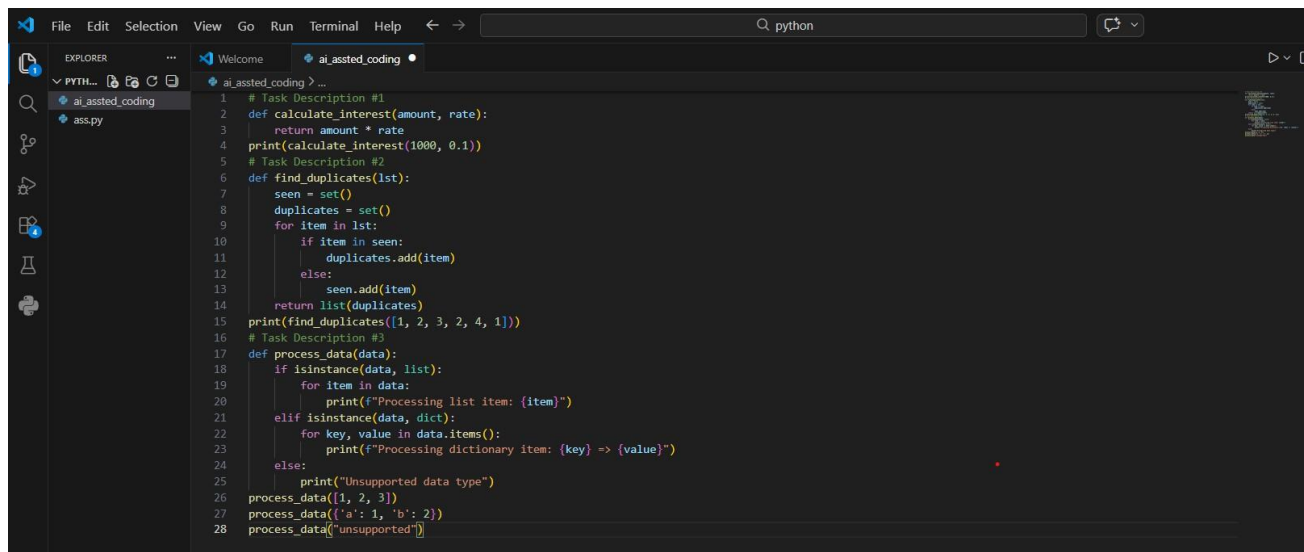
```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

Code:

A screenshot of a code editor window with a dark theme. The Explorer sidebar on the left shows a file named 'ai_assted_coding.py'. The main editor area displays the following Python code:

```
1 # Task Description #1
2 def calculate_interest(amount, rate):
3     return amount * rate
4 print(calculate_interest(1000, 0.1))
5 # Task Description #2
6 def find_duplicates(lst):
7     seen = set()
8     duplicates = set()
9     for item in lst:
10         if item in seen:
11             duplicates.add(item)
12         else:
13             seen.add(item)
14     return list(duplicates)
15 print(find_duplicates([1, 2, 3, 2, 4, 1]))
16 # Task Description #3
17 def process_data(data):
18     if isinstance(data, list):
19         for item in data:
20             print(f"Processing list item: {item}")
21     elif isinstance(data, dict):
22         for key, value in data.items():
23             print(f"Processing dictionary item: {key} => {value}")
24     else:
25         print("Unsupported data type")
26 process_data([1, 2, 3])
27 process_data({'a': 1, 'b': 2})
28 process_data("unsupported")
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
100.0
[1, 2]
Processing list item: 1
Processing list item: 2
Processing list item: 3
Processing dictionary item: a => 1
Processing dictionary item: b => 2
Unsupported data type
PS C:\Users\anapu\Downloads\python> █
```

Observation:

The code demonstrates basic Python functions to perform different tasks such as calculating interest, finding duplicate elements in a list, and processing different data types. It uses fundamental concepts like loops, sets, conditional statements, and type checking to handle and process data efficiently.

Task Description #2 : (Refactoring Deeply Nested Conditionals)

Sample Input & Prompt:

Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

• Focus Areas:

- o Readability
- o Logical simplification
- o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

```
print("Good")
```

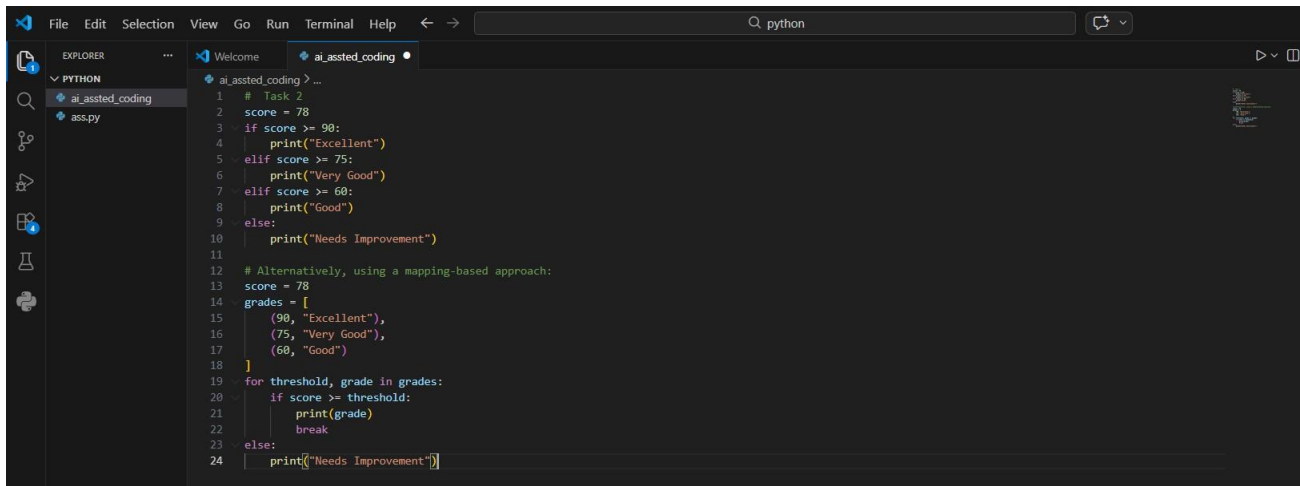
else:

```
print("Needs Improvement")
```

Expected Outcome:

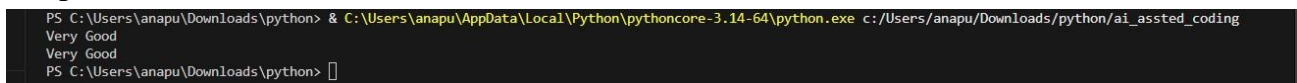
- o Flattened logic using guard clauses or a mapping-based approach.

Code:



```
1 # Task 2
2 score = 78
3 if score >= 90:
4     print("Excellent")
5 elif score >= 75:
6     print("Very Good")
7 elif score >= 60:
8     print("Good")
9 else:
10    print("Needs Improvement")
11
12 # Alternatively, using a mapping-based approach:
13 score = 78
14 grades = [
15     (90, "Excellent"),
16     (75, "Very Good"),
17     (60, "Good")
18 ]
19 for threshold, grade in grades:
20     if score >= threshold:
21         print(grade)
22         break
23 else:
24     print("Needs Improvement")
```

Output:



```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Very Good
Very Good
PS C:\Users\anapu\Downloads\python>
```

Observation:

The code classifies a student's performance based on the score using conditional statements. It also shows an alternative approach using a list of thresholds and grades to determine the result more flexibly.

Task 3 (Refactoring Repeated File Handling Code)

Sample Input & Prompt:

- **Task:** Use AI to refactor repeated file open/read/close logic.
- **Focus Areas:**
 - o DRY principle
 - o Context managers
 - o Function reuse

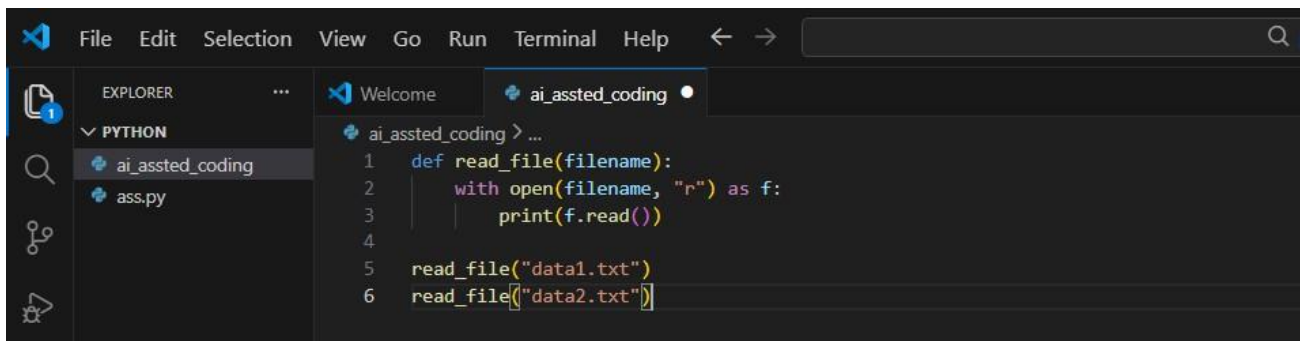
Legacy Code:

```
f = open("data1.txt")  
print(f.read()) f.close()  
  
f = open("data2.txt")  
print(f.read()) f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.

Code:

A screenshot of a code editor interface. The Explorer panel on the left shows a project named 'ai_assted_coding' containing a file 'ass.py'. The main editor area shows the following Python code:

```
1 def read_file(filename):  
2     with open(filename, "r") as f:  
3         print(f.read())  
4  
5 read_file("data1.txt")  
6 read_file(["data2.txt"])
```

Output:

Hello from file 1

Observation:

The code defines a function to **read and return the contents of a file** using the `with open()` statement.

It then reads and prints the contents of **two files**: `data1.txt` **and** `data2.txt`.

Task 4 (Optimizing Search Logic)

Sample input & Prompt :

Task: Refactor inefficient linear searches using appropriate data structures.

• Focus Areas:

- o Time complexity
- o Data structure choice

Legacy Code:

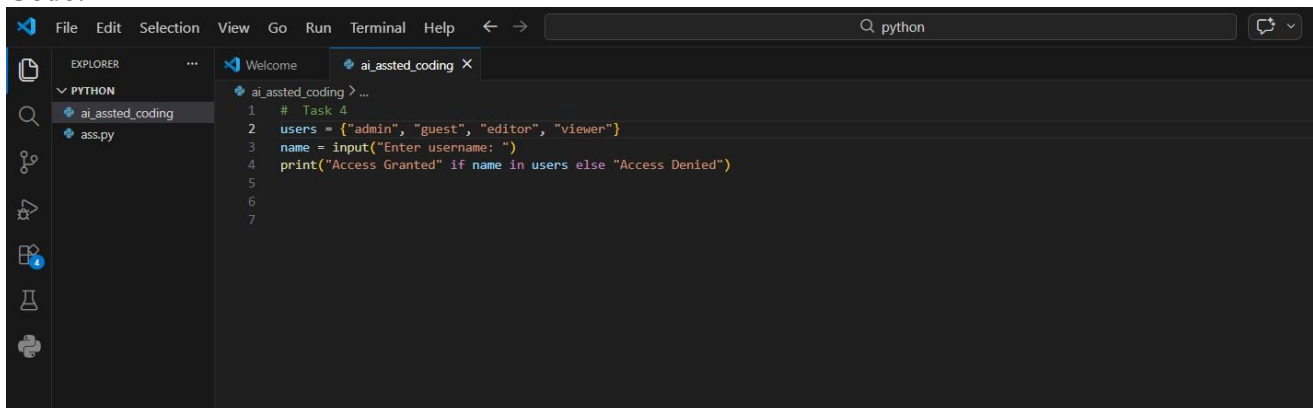
```
users = ["admin", "guest", "editor", "viewer"]
```

```

name = input("Enter username: ")
found = False
for u in users:
    if u == name:
        found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:
o Use of sets or dictionaries with complexity justification.

```

Code:



The screenshot shows a Visual Studio Code editor with a dark theme. The Explorer panel on the left shows a file named 'ai_assted_coding.py'. The main editor area displays the following Python code:

```

1 # Task 4
2 users = {"admin", "guest", "editor", "viewer"}
3 name = input("Enter username: ")
4 print("Access Granted" if name in users else "Access Denied")
5
6
7

```

Output:

```

PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter username: abhi
Access Denied
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter username: admin
Access Granted

```

Observation:

The code checks whether the entered username exists in the predefined set of users. It uses a conditional expression to display “**Access Granted**” if the user exists, otherwise “**Access Denied.**”

Task 5 (Refactoring Procedural Code into OOP Design)

Sample input & Prompt:

- **Task:** Use AI to refactor procedural code into a class-based design.

• Focus Areas:

- o Object-Oriented principles
- o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

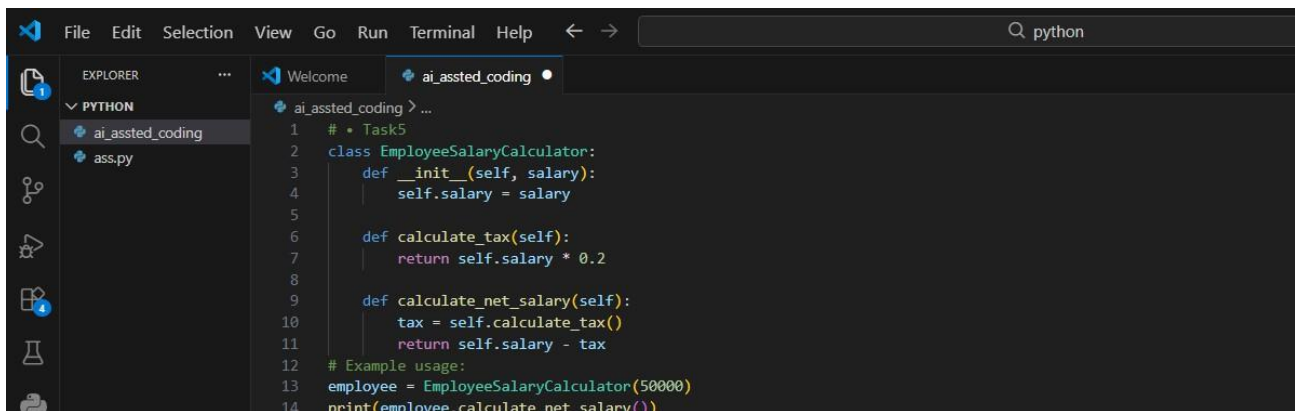
net = salary - tax

print(net)

Expected Outcome:

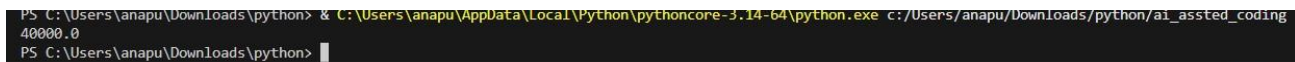
- o A class like EmployeeSalaryCalculator with methods and attributes.

Code:

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'ai_assted_coding' with a file 'ass.py'. The main editor window displays the code for 'ai_assted_coding'. The code defines a class 'EmployeeSalaryCalculator' with an '__init__' method that takes 'salary' as an argument and assigns it to 'self.salary'. It also has two methods: 'calculate_tax' which returns 'self.salary * 0.2', and 'calculate_net_salary' which calls 'calculate_tax' and returns 'self.salary - tax'. An example usage is provided at the bottom, creating an instance of the class with a salary of 50000 and printing the result of 'calculate_net_salary()'.

```
1 # * Task5
2 class EmployeeSalaryCalculator:
3     def __init__(self, salary):
4         self.salary = salary
5
6     def calculate_tax(self):
7         return self.salary * 0.2
8
9     def calculate_net_salary(self):
10        tax = self.calculate_tax()
11        return self.salary - tax
12
13 # Example usage:
14 employee = EmployeeSalaryCalculator(50000)
15 print(employee.calculate_net_salary())
```

Output:

A screenshot of a terminal window. The first line shows the command to run the Python script: 'PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding'. The second line shows the output of the script: '40000.0'. The third line shows the prompt 'PS C:\Users\anapu\Downloads\python>' with a cursor.

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
40000.0
PS C:\Users\anapu\Downloads\python> |
```

Observation:

The code defines a **class** to calculate an employee's salary after deducting tax. It calculates **20% tax from the salary and returns the remaining amount as the net salary.**

Task 6 (Refactoring for Performance Optimization)

Sample Input & Prompt:

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- Algorithmic optimization
- Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

```
    if i % 2 == 0:
```

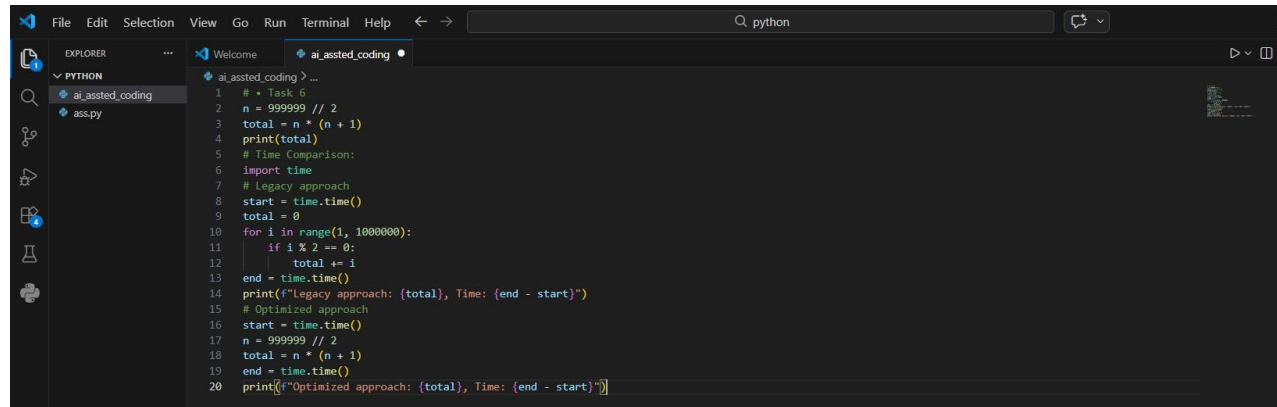
```
        total += i
```

```
print(total)
```

Expected Outcome:

- Optimized logic using mathematical formulas or comprehensions, with time comparison.

Code:



```
1 # Task 6
2 n = 999999 // 2
3 total = n * (n + 1)
4 print(total)
5 # Time Comparison:
6 import time
7 # Legacy approach
8 start = time.time()
9 total = 0
10 for i in range(1, 1000000):
11     if i % 2 == 0:
12         total += i
13 end = time.time()
14 print(f"Legacy approach: {total}, Time: {end - start}")
15 # Optimized approach
16 start = time.time()
17 n = 999999 // 2
18 total = n * (n + 1)
19 end = time.time()
20 print(f"Optimized approach: {total}, Time: {end - start}")
```

Output:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
249999500000
Legacy approach: 249999500000, Time: 0.08654546737670898
Optimized approach: 249999500000, Time: 1.6689300537109375e-06
PS C:\Users\anapu\Downloads\python> █
```

Observation:

The code compares a **legacy loop-based approach** and an **optimized mathematical formula** to calculate the sum of even numbers up to 1,000,000.

The optimized method is faster because it directly uses a formula instead of iterating through each number.

Task 7 (Removing Hidden Side Effects)

Sample Input & Prompt:

- **Task:** Refactor code that modifies shared mutable state.

- **Focus Areas:**

- Functional-style refactoring

- Predictability

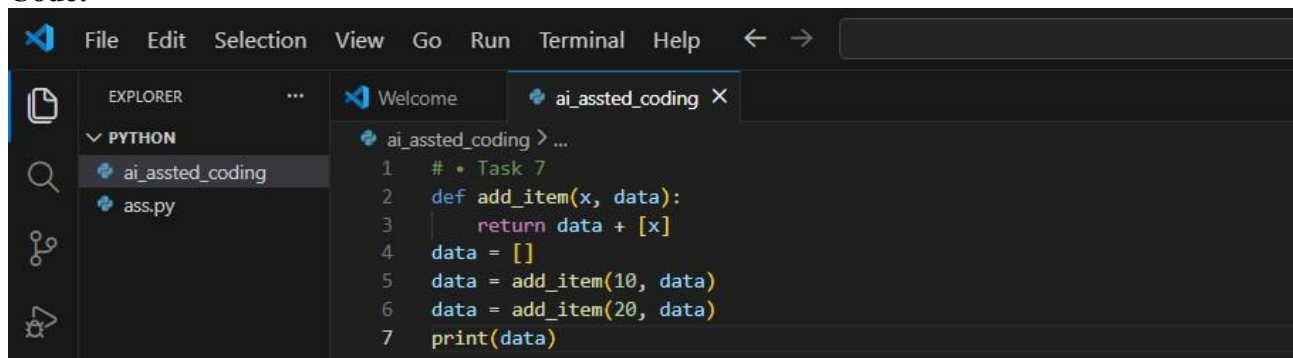
Legacy Code:

```
data = []
def add_item(x):
    data.append(x)
add_item(10)
add_item(20)
print(data)
```

Expected Outcome:

- Refactored function returning values instead of mutating globals

Code:



```
File Edit Selection View Go Run Terminal Help
EXPLORER
PYTHON
  ai_assted_coding
  ass.py
ai_assted_coding > ...
1 # • Task 7
2 def add_item(x, data):
3     return data + [x]
4 data = []
5 data = add_item(10, data)
6 data = add_item(20, data)
7 print(data)
```

Output:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
[10, 20]
PS C:\Users\anapu\Downloads\python> |
```

Observation:

The code defines a function that adds a new item to a list and returns the updated list. It demonstrates how values can be appended by creating a new list instead of modifying the original list directly.

Task 8 (Refactoring Complex Input Validation Logic)

Sample Input & Prompt:

- **Task:** Use AI to simplify and modularize complex validation

rules.

- **Focus Areas:**

- Readability

- Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
if any(c.isdigit() for c in password):
```

```
if any(c.isupper() for c in password):
```

```
print("Valid Password")
```

```
else:
```

```
print("Must contain uppercase")
```

```
else:
```

```
print("Must contain digit")
```

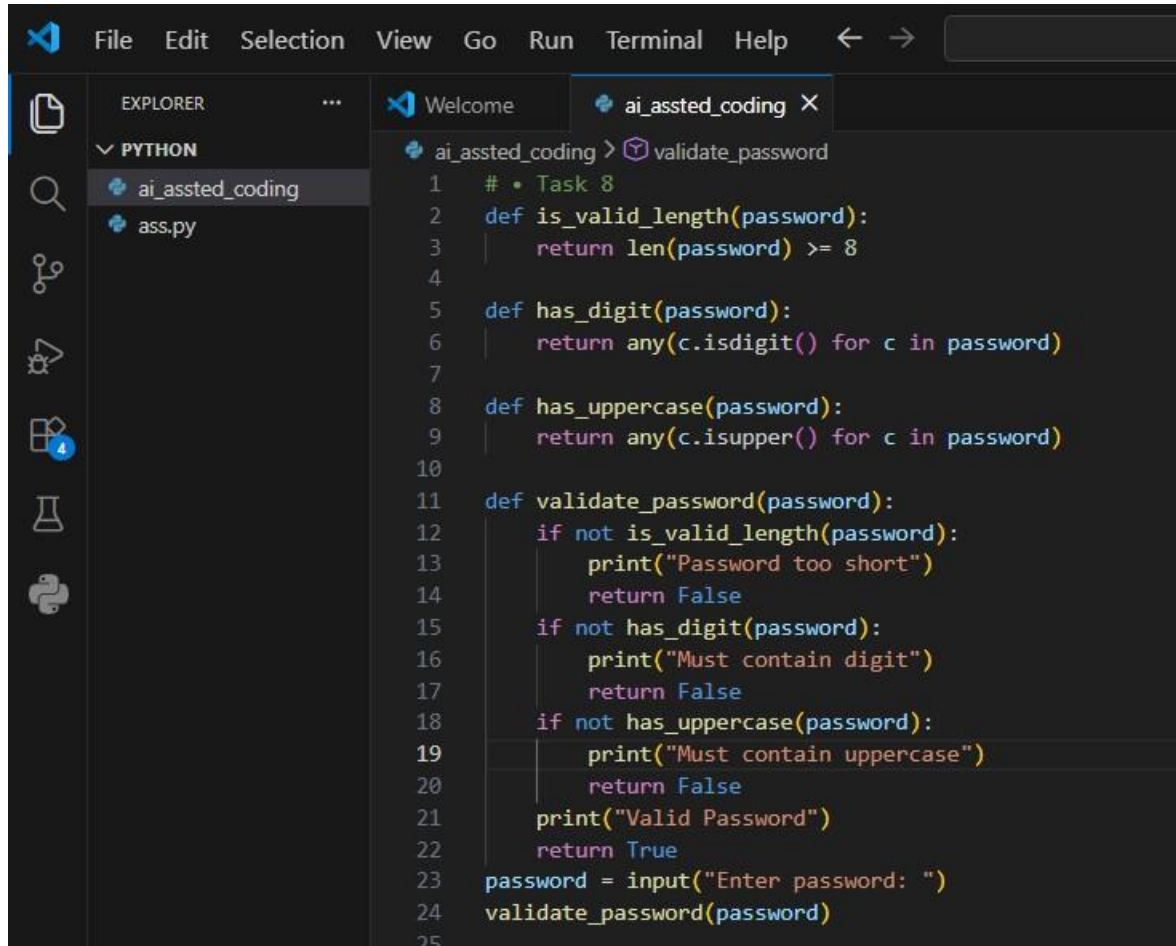
```
else:
```

```
print("Password too short")
```

Expected Outcome:

- o Separate validation functions with clear responsibility

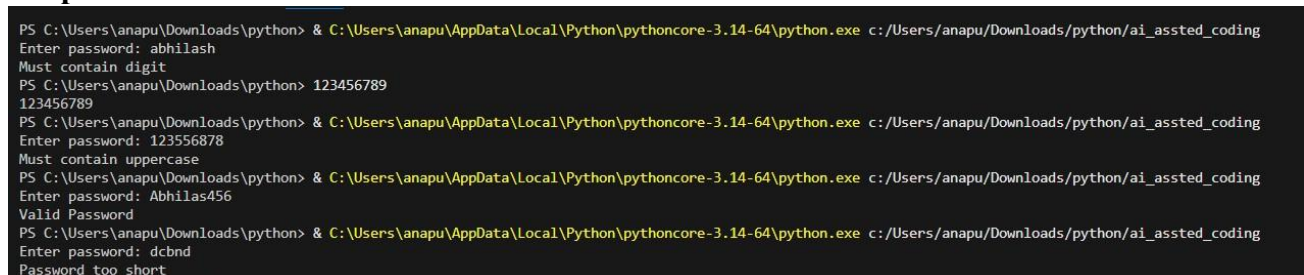
Code:



The screenshot shows the Visual Studio Code editor with a file named `ai_assted_coding` open. The file contains a Python script for password validation. The script defines three helper functions: `is_valid_length`, `has_digit`, and `has_uppercase`. These functions are used within a `validate_password` function that checks the password against these criteria and prints appropriate messages. The script also includes a main loop that takes user input and calls the `validate_password` function.

```
1 # • Task 8
2 def is_valid_length(password):
3     return len(password) >= 8
4
5 def has_digit(password):
6     return any(c.isdigit() for c in password)
7
8 def has_uppercase(password):
9     return any(c.isupper() for c in password)
10
11 def validate_password(password):
12     if not is_valid_length(password):
13         print("Password too short")
14         return False
15     if not has_digit(password):
16         print("Must contain digit")
17         return False
18     if not has_uppercase(password):
19         print("Must contain uppercase")
20         return False
21     print("Valid Password")
22     return True
23 password = input("Enter password: ")
24 validate_password(password)
25
```

Output:



The screenshot shows a terminal window with the following output:

```
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter password: abhilash
Must contain digit
PS C:\Users\anapu\Downloads\python> 123456789
123456789
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter password: 123556878
Must contain uppercase
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter password: Abhilas456
Valid Password
PS C:\Users\anapu\Downloads\python> & C:\Users\anapu\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/anapu/Downloads/python/ai_assted_coding
Enter password: dcbnd
Password too short
```

Observation:

The code validates a password by checking its length, presence of a digit, and an uppercase letter using separate functions. It prints appropriate messages and confirms **“Valid Password”** only when all conditions are satisfied. □

